Assessment Cover Sheet 2025/26

Module code and title	6G6Z0014 – Mobile Computing
Assessment set by	Adrian Davison and Adam Miles
Assessment ID	1CWK100
Assessment weighting	100%
Assessment title	Developing a Cross-Platform Mobile App
Type	Individual
Hand-in deadline	1st May 2026, 21:00
Hand-in format and mechanism	Upload to Moodle

Learning outcomes being assessed

LO1: Create application capable of running on contemporary mobile devices using appropriate tools and techniques.

LO2: Utilise specialist on-board mobile hardware by interfacing with appropriate APIs and libraries.

Note: it is your responsibility to make sure that your work is complete and available for marking by the deadline. Make sure that you have followed the submission instructions carefully, and your work is submitted in the correct format, using the correct hand-in mechanism (e.g., Moodle upload). If submitting via Moodle, you are advised to check your work after upload, to make sure it has uploaded properly. If submitting via OneDrive, ensure that your tutors have access to the work. **Do not alter your work after the deadline**. You should make at least one full backup copy of your work.

Penalties for late submission

The timeliness of submissions is strictly monitored and enforced.

All coursework has a late submission window of 7 calendar days, but any work submitted within the late window will be capped at 40%, unless you have an agreed extension. Work submitted after the 7-day late window will be capped at zero unless you have an agreed extension. See 'Assessment Mitigation' below for further information on extensions.

Please note that individual tutors are unable to grant any extensions to assessments.

Assessment Mitigation

If there is a valid reason why you are unable to submit your assessment by the deadline you may apply for Assessment Mitigation. There are two types of mitigation you can apply for via the module area on Moodle (in the 'Assessments' block on the right-hand side of the page):

- **Non-evidenced extension**: does **not** require you to submit evidence. It allows you to add a **short** extension to a deadline. This is not available for event-based assessments such as in-class tests, presentations, interviews, etc. You can apply for this extension during the assessment weeks, and the request must be made **before** the submission deadline. For this assessment, the non-evidenced extension is 2 days.
- **Evidenced extension**: requires you to provide independent evidence of a situation which has impacted you. Allows you to apply for a longer extension and is available for event-based assessment such as in-class test, presentations, interviews, etc. For event-based assessments, the normal outcome is that the assessment will be deferred to the summer reassessment period.

Further information about Assessment Mitigation is available on the dedicated Assessments page.

Personal Learning Plans (PLP)

If you have a Personal Learning Plan (PLP) which states you can negotiate an extended deadline, submit an evidenced extension request on the Moodle site for the module.

Plagiarism

Plagiarism is the unacknowledged representation of another person's work, or use of their ideas, as one's own. Manchester Metropolitan University takes care to detect plagiarism, employs plagiarism detection software, and imposes severe penalties, as outlined in the Student Code of Conduct and Academic integrity and misconduct. Poor referencing or submitting the wrong assignment may still be treated as plagiarism. If in doubt, seek advice from your tutor.

As part of a plagiarism check, you may be asked to attend a meeting with the Module Leader, or another member of the module delivery team, where you will be asked to explain your work (e.g. explain the code in a programming assignment). If you are called to one of these meetings, it is very important that you attend.

Use of generative AI

Not permitted

The use of generative AI is not permitted for this assessment. This includes but is not limited to tools like Microsoft Copilot or ChatGPT (please see What is Artificial Intelligence? if you are not sure). All submitted work must be your own original content, created without the aid of generative AI. Failure to adhere to this policy may constitute academic misconduct and result in serious consequences. If you have been advised to use a specific tool as part of a personal learning plan, you should continue to use it.

If you are unable to upload your work to Moodle

If you have problems submitting your work through Moodle, you can raise a ticket with the Assessment Management Team using the Assist Portal. This must be done **before the published deadline**, else your work will be logged as a late submission. Alternatively, you can save your work into a single zip folder then upload the zip folder to your university OneDrive and submit a Word document to Moodle which includes a link to the folder. **It is your responsibility to make sure you share the OneDrive folder with the Module Leader, or it will not be possible to mark your work.**

Assessment Regulations

For further information see the Undergraduate Assessment Regulations on the Assessments and Results information pages

Formative feedback:	Verbal feedback provided in timetabled sessions and during office hours.
Summative feedback:	Summative feedback will be given after the submission of the below assignment and based on the stated criteria.

Assessment Brief

Students will be tasked with the development of a mobile application, utilising the .NET Multi-platform App User Interface (.NET MAUI) framework for the display of data, and making use of the mobile device's hardware for suitable additional functionality.

You will need to commit your code to your designated GitHub repository regularly. The final submission will be assessed using the following criteria. Each element of the criteria will be explained by you in the form a screencast. Code quality and GitHub usage will be checked by the tutor, but you should also mention these in your screencast.

What should my App be?

Your app can be anything you wish and could align to your own interests. Some examples could be a workout planner for health and fitness, a skills development app for programming within an education theme, or a travel app that can include a city guide and use GPS to find useful landmarks near you. These are just examples. With this flexibility of choice, you can create something unique. More examples will be posted to Moodle. You can also explore the common app stores for inspiration: Google's Play store and Apple's App Store.

Development Software and Use of Macs

In the labs, we will use Visual Studio 2022 on Windows machines with .NET MAUI using .NET 9.0. The labs provide everything needed to develop your assessment. If you use your own Windows machine, please try to match up versions with the lab to avoid any software mismatches and to align with the support provided.

As we do not use Apple Mac machines, we have limited support for people using their own Mac/MacBook, so you will be expected to understand your own personal machine and set up accordingly. Visual Studio is no longer supported by Macs, so you will need to use Visual Studio Code on your personal machine if you decide to keep using your Mac for development.

Another alternative IDE is JetBrains Rider, which can work on both Windows and Mac. This is not set up in the university labs, so you can only use it on your personal machine. It's free for non-commercial use and as a student, you can sign up for the JetBrains student pack here.

We can help the most with lab machines as they are set up for the module. If you decide to use your personal machine, be aware that it is your responsibility to set up everything correctly.

Submission Criteria

1. UI/UX Design and Accessibility

You should create a good user interface (UI) and user experience (UX). The UI should be simple and kept uncluttered to avoid overwhelming users. You should be consistent in your design, such as with the use of colours, fonts, and icons. eXtensible Application Markup Language (XAML) should be used for defining UIs.

For the UX, the navigation should be intuitive and straightforward. A good idea is to use elements you see every day in mobile applications, such as arrows for going back or a question mark (?) for accessing help. The performance of the app should also be as optimised as possible to improve the overall experience when using the app, for example, minimising load times and reusing code as much as possible.

Accessibility in mobile computing is essential to ensure that all users, including those with disabilities, can

effectively interact with mobile applications. In a lot of cases, it will be a legal requirement for apps that are published. This criterion will be assessed on how much consideration is put into applying best accessibility practices, for example: ensuring font sizes are readable and changeable, the ability to turn on dark mode or similar, and screen readers.

While there is not a set amount of accessibility features to include in the assignment, it is recommended that you research the Web Content Accessibility Guidelines (WCAG 2 Overview | Web Accessibility Initiative (WAI) | W3C) to understand the principles and how to meet them in your app. You will then need to ensure you understand and talk through what you have applied in your final screencast.

2. Use of Mobile Hardware

Your app will need to utilise the specialist hardware available on mobile devices, such as the camera or accelerometer. There will be different marks given based on the number of hardware features used (see in depth mark scheme below). It is expected that the hardware feature should work for this to count. For higher marks (75%+), it will be expected to use the hardware features in more unique ways. For example, using the camera for a computer vision application that classifies an image.

The following is a list of hardware features that **would** count towards this criterion (**please ask a tutor if you are unsure if something is a feature**):

- Camera (using both the front and back camera only counts as 1 feature)
- Microphone
- Accelerometer
- Gyroscope
- Magnetometer
- Barometer
- Shake
- Compass
- Flash (needs to be implemented in code, **not** just used as the in-built function on the camera)
- Location/geolocation/geocoding (can be spoofed in an emulator rather than a real location)
- Haptic feedback and vibration (using both only counts as 1 feature)
- Text-to-speech

The following is a list of hardware features that **would not** count towards this criterion (**please ask a tutor if you are unsure if something is not a feature**):

- Battery level
- Orientation
- Device information
- Buttons or any other UI element
- Speaker
- Touch screen gestures
- Storage

- Databases

The above lists may not be exhaustive, so please ask if you are unsure.

3. Functionality

The functionality of the app will be assessed here, for example, if the buttons created go to the right place, the menus work, and you are able to show all this in your screencast. Other criteria may link to this, for example if you have a camera element to take a picture and it works well, this will also go towards this criterion. If your app should work in any orientation, such as portrait and landscape, then this should be demonstrated as part of your app's functionality. If your app is intended to only work in one type of orientation, then make sure to state this in your screencast.

4. Validation and Error Handling

Part of this criterion is to check how you handle user input correctly. So, if you have a login page, you will need to ensure that a user enters the information and does not leave a box blank. If they do, you need to provide a validation check and provide a response to the user to correct the mistake.

In addition, your code should show evidence of handling errors. In your screencast, you may walk through elements of your code, or you can input data that will throw an error and show how you handle it without the app crashing. All code should have some form of error handling as best practice, so ensure you show it off during the screencast. Not doing so will mean this criterion will be scored low. You do not need to do unit testing but make sure to demonstrate error handing and validation in some way.

5. Code Quality

The quality of your code is how well-written and maintainable your code is. The higher the quality of code will make it easier to work with, less likely to have bugs, and can be maintained over longer periods of time by yourself or others.

High-quality code will perform intended tasks correctly and efficiently; it should be readable and easy to understand with proper naming conventions and detailed comments. It should be consistent and follow coding standards and best practices (for this assessment, it will be for .NET/C#). You should also use common techniques, such as code reuse, to improve the quality of your code.

6. Deployment

As we are working in a cross-platform framework, you should be able to show evidence that your app can compile and run on multiple device types. You will need to show that you can deploy to Android and at least 1 other device (Windows, iOS, tablets). This can be through emulators or physical devices. Due to the university labs only running Windows, **we cannot deploy to iOS in the labs directly**. However, you are welcome to try to deploy your code on an iOS device if you have the means to do so. It will count towards a deployment, but it is **not a requirement to deploy to iOS**.

An example of deploying to just 1 Android device would be:

- An Android phone emulator
- An Android phone physical device
- An Android tablet

To score higher marks, you will need to deploy to an Android device and at least 1 other type. Some examples are:

- An Android phone emulator **AND** an Android tablet emulator
- A physical Android phone **AND** an Android tablet emulator
- An Android phone emulator **AND** an iOS phone emulator

If you use a physical Android phone and an Android phone emulator, this will **only count as 1**.

7. GitHub Usage

You will be assigned a GitHub repository (repo) that will be set up by your tutor. There will be a very basic sample project to begin with that you can modify and work on. You can also completely push your own project if you prefer. You will need to sign up to GitHub if you do not have an account.

You must ensure the most recent version of your work is committed or published on GitHub before the deadline. **Any work committed after your deadline will not be considered towards the mark of this assessment**. Instead, your tutors will revert to the project's latest version before the deadline. As part of the assessment, you must demonstrate good working knowledge and continuous usage of GitHub, **not single-push repos**.

To get the highest marks, you need to regularly commit to your repository. You should commit little and often. For example, if you are committing over 200+ lines of code in one commit, you likely need to break your commits down more. We also do not expect you to submit every tiny change, such as spelling mistakes or changing a single variable name. A good rule of thumb is to commit when you add something like a new function or UI element.

Submission of Deliverables

You are required to submit the following to Moodle:

1. A link to the GitHub repository where you are expected to make regular commits of your code to keep it updated and show your progress. These repositories will be assigned to you through GitHub classroom. The GitHub repository should also contain a README file that describes your application and other information (see detailed mark scheme).
2. A screencast of up to **15 minutes** explaining how you meet each of the criteria set out above and below. This screencast will be uploaded through **mmutube**. This upload can then be 'embedded' to Moodle's submission area (this avoids large file size uploads). A video guide on how this is done can be found here.
    a. Your screencast maximum length should be **15 minutes**. If you go over this by **1 or more minutes** you will have a **video overtime penalty** deducted from your final mark of **5%.**

IMPORTANT: The code will **NOT** be run by the tutor marking your work. You need to ensure that you talk through everything in your app that meets the marking criteria, as the screencast will be the main marking source. If you fail to submit a screencast, you **will be marked 0**. Your code quality and GitHub usage is marked outside of the screencast, but the screencast and code are **REQUIRED** to be classed as a submission.

Marking Scheme (out of 100%)

Brief Mark Scheme

UI/UX Design and Accessibility	**30%**
Use of mobile hardware	**20%**
Functionality	**20%**
Validation and Error Handling	**10%**
Code Quality	**10%**
Deployment	**5%**
GitHub Usage	**5%**

Detailed Mark Scheme

The indicative criteria that will be applied to each part of the submission are given in the table below. Most elements are assessed through the screencast, so aim to explain all these during your recording. GitHub usage and code quality will be assessed through your assigned GitHub repository.

IMPORTANT: Your screencast maximum length should be **15 minutes**. If you go over this by 1 **or more minutes** you will have a **video overtime penalty** deducted from your final mark of **5%.**

Department of Computing and Mathematics

Manchester Metropolitan University

Grade Band	Fail (0-19%)	Fail (20-39%)	3rd (40-49%)	2:2 (50-59%)	2:1 (60-69%)	1st (70-85%)	1st (86-100%)
UI/UX Design and Accessibility (30%)	**UI/UX Design** • Little to no consideration to usability and design. • Little to no use of XAML. • App performance makes it unusable. **Accessibility** • Very little application of accessibility features. • No reference to WCAG. • No user instructions.	**UI/UX Design** • Little consideration to usability and design. • Very limited XAML use on 1 page. • App performance is poor. **Accessibility** • Little application of accessibility features. • Little reference or alignment to WCAG. • Partial user instructions available.	**UI/UX Design** • Some consideration to usability and design. • Very limited XAML use across multiple pages. • App performance is not well optimised. **Accessibility** • Some consideration to accessibility features. • Does not clearly align to WCAG. • Simple user instructions available.	**UI/UX Design** • Good usability practices and style but may not provide the smoothest experience for users. • Some use of XAML. • App performance is somewhat optimised. **Accessibility** • Good application of accessibility features but does not apply more than half. • Basic alignment to WCAG. • User instructions available but not always clear.	**UI/UX Design** • Good use of style, usability, and frameworks. • Good use of XAML. • App performance is well optimised but may be the smoothest experience. **Accessibility** • Good application of accessibility features. • Good alignment and reference to WCAG. • User instructions are available and are mostly clear.	**UI/UX Design** • Great use of style, usability, and frameworks. • Great use of XAML. • App performance is mostly smooth and optimised. **Accessibility** • Great application of accessibility features. • Mostly follows and references WCAG. • Clear user instructions are available with text and no/few images.	**UI/UX Design** • Exceptional and consistent style, usability, and use of frameworks. • Extensive use of XAML. • App performance is very well optimised. **Accessibility** • Full application of accessibility features aligned to relevant guidelines. • Strictly adheres to and references WCAG for mobile applications. • Clear user instructions are available with images at all relevant steps.
Use of mobile hardware (20%)	• Uses no specialist mobile hardware.	• Mobile hardware feature attempted but does not correctly work.	• Demonstrates the use of a minimum of 1 different mobile hardware features.	• Demonstrates the use of a minimum of 2 different mobile hardware features.	• Demonstrates the use of a minimum of 3 different mobile hardware features.	• Demonstrates the use of a minimum of 4 different mobile hardware features.	• Demonstrates the use of a minimum 4 different mobile hardware features and uses advanced methods, such as

							machine learning and computer vision, alongside mobile hardware.
Functionality (20%)	• App uses little to no basic functionality (e.g., buttons, dynamic text, orientation handling).	• App uses little basic functionality (e.g., buttons, dynamic text, orientation handling).	• App uses simple basic functionality (e.g., buttons, dynamic text, orientation handling).	• App uses some basic functionality (e.g., buttons, dynamic text, orientation handling) and some advanced functionality is attempted (e.g., gestures and different menu styles).	• App uses a large amount basic functionality elements and some advanced elements.	• App uses of a large amount basic functionality elements and advanced elements.	• Use of many different basic and advanced functionality elements (e.g., buttons, gesture control, networking etc.) and each element works together seamlessly (e.g., button presses updating a graph).
Validation and Error Handling (10%)	• Very little to no validation. • Errors are not handled. • No messages shown to the user.	• Little validation used. • Little error handling used. • Few messages shown to the user.	• Some validation was used but may not work correctly. • Some error handling used but may not work correctly. • Some messages shown to the user.	• The app has good validation to minimise errors, mostly for user input. • App has good error handling, mostly for user input. • Messages shown to the user are sometimes related to error and validation handling.	• The app has good validation to minimise errors due to user input and other issues. • Many errors are handled correctly. • Error and validation messages shown to the user but are not user friendly.	• The app has great validation to minimise errors due to user input and other issues. • Most errors are handled correctly. • Error and validation messages shown to the user but may not be very clear.	• The app has robust validation to minimise errors due to user input and other issues. • All errors are handled correctly. • Error and validation messages shown to the user are clear, informative, and user friendly.
Code Quality (10%)	• Extremely poorly formatted or no code. • No comments or no code.	• Poorly formatted. • Very few comments. • Partially logical code structure.	• Partial formatting. • A few comments. • Partial code structure organisation, with	• Good formatting. • Good comments. • Satisfactory code structure organisation, with	• Very good formatting. • Many good comments. • Very good code	• Great formatting. • Mostly commented. • Great code structure organisation, with functions, classes,	• Extremely well formatted. • Fully commented. • Code structure organised logically

	• No logical code structure or no code. • No variable naming conventions or no code.	• Very few variable naming conventions.	few functions but no classes. • A few variable naming conventions.	few functions but no classes. • Some variable naming conventions.	structure organisation, with few functions and classes. • Many variable naming conventions.	and evidence of reusability. • Most variable naming conventions.	with functions, classes, reusability and evidence of MVVM structure. • All variables named clearly and consistently.
Deployment (5%)	• App can load on 1 emulator that is either Android-based or another device (Windows, tablets, iOS etc.). • Loaded app cannot be navigated around. • No hardware features implemented across any platform.	• App can load on 1 emulator or physical device that is Android-based and another device (Windows, tablets, iOS etc.). • Loaded app can be navigated around using the emulator or physical device. • Hardware features implemented but do not work the same across demonstrated platforms.	• App can load on Android and another device (Windows, tablets, iOS etc.) emulator/physical device with large issues on native functionality in at least 1 device type. • Loaded app can be navigated around but does not run smoothly. • A few hardware features work the same across demonstrated platforms.	• Can be deployed to Android and another device (Windows, tablets, iOS etc.), with major issues on native functionality in 1 device type. • Loaded app can be navigated around with basic features available. • Some hardware features work the same across demonstrated platforms.	• Can be deployed to Android and another device (Windows, tablets, iOS etc.) with minor issues on native functionality in both device types. • Loaded app can be navigated around, runs smoothly, with basic features available. • Many hardware features work the same across demonstrated platforms.	• Can be deployed to Android and another device (Windows, tablets, iOS etc.) with minor issues on native functionality in 1 device type. • Loaded app can be used smoothly with all advanced features available. • Most hardware features work the same across demonstrated platforms.	• Can be deployed to Android and another device (Windows, tablets, iOS etc.) with no issues. • Loaded app works the same across Android and another device (Windows, tablets, iOS etc.) with all advanced features available. • All hardware features work the same across demonstrated platforms.
GitHub Usage (5%)	• No code submitted to GitHub or single commit. • No commit messages. • No README file or not changed from	• GitHub commit is not correct e.g., does not correspond to app shown in screencast. • Commit messages exist but are not useful.	• The GitHub repo has a few useful commits to the project. • Commit messages exist but lack detail. • Simple README file exists with	• GitHub is nicely organised. • Repo has good commit messages that are simple but understandable. • README file exists with author	• GitHub is nicely organised and used regularly. • Commit messages are detailed and show development progress.	• GitHub organised very well and used very regularly. • Repo has useful and detailed commit messages showing development progress and fixes. • README contains a	• A professional standard GitHub repo. • Commits are very regular with helpful commit messages showing development over an extended period.

	the template.	• Partially complete README file included.	author's name and app title.	name, app title, and basic information on the app.	• README file exists with author name, app title, and detailed information on the app.	brief development plan (i.e., what features to include) and previous information.	• Detailed README with all previous information and anything additional that helps with running the app.